Inspiring Excellence

SUMMER 2026

CSE729: BLOCKCHAIN AND DISTRIBUTED LEDGER

Name (ID): **Suprava Saha Dibya (21201808)**
Ahmed Sayef (21201220)

Section: **1**

Project: **2**

Date of Submission: **September 10, 2026**

Abstract

StreamPay is a decentralized salary streaming protocol designed to replace traditional fixed payroll cycles with continuous blockchain-based salary distribution. The system allows employers to deposit ETH into smart contracts and release employee compensation gradually according to elapsed time.

The project implements three major roles: Protocol Administrator, Company (Employer), and Employee. Employers can register employees and create salary streams. Employees can observe real-time vesting progress and withdraw earned funds. The administrator manages accumulated protocol fees generated from salary settlements.

The implementation combines Solidity smart contracts, Foundry development tools, Anvil local blockchain infrastructure, MetaMask wallet authentication, and a React-based decentralized application.

Github Repository

https://github.com/suprava-ssd/cse729_project-2

1. Introduction

Traditional salary systems usually operate through monthly or weekly payment schedules. Employees complete work before receiving compensation, creating delays between performed work and received income.

Blockchain technology enables programmable financial agreements where payment rules can be automatically enforced by smart contracts. StreamPay applies this concept to salary streaming by creating time-based salary agreements.

Instead of transferring the complete salary immediately, the employer locks funds inside a smart contract. The employee earns access to the salary continuously as time passes.

2. Problem Statement

The project addresses several limitations of conventional payroll systems:

- Employees cannot access earned income before the scheduled payroll date.
- Payroll processing depends on centralized systems.
- Payment transparency is limited.
- Manual salary settlement can create disputes.

StreamPay solves these issues by introducing an automated, transparent, and blockchain-enforced salary distribution mechanism.

3. Project Objectives

The main objectives are:

1. Create a decentralized salary streaming protocol.
2. Implement Ethereum-based salary vesting.
3. Allow employers to manage employee relationships.
4. Provide employees with real-time salary visibility.
5. Enable secure withdrawals.
6. Implement protocol fee collection.
7. Support stream cancellation with fair settlement.
8. Validate the complete workflow through automated testing.

4. System Architecture

StreamPay follows a layered decentralized architecture.

- **User Layer:** Admin, Employer, and Employee interact with the application.
- **Wallet Layer:** MetaMask manages accounts and signs blockchain transactions.
- **Frontend Layer:** The React application communicates with the smart contract.
- **Smart Contract Layer:** The Solidity contract manages companies, employees, streams, withdrawals, fees, and cancellation.
- **Blockchain Layer:** Anvil provides a local Ethereum-compatible testing environment.

5. Technology Stack

StreamPay uses a combination of blockchain development, wallet integration, and frontend technologies as shown in Figure 1.

- **Solidity** is used to implement the StreamPay smart contract and define the core salary streaming and settlement logic.
- **Foundry** is used as the primary development framework for compiling, deploying, and automatically testing the smart contract.
- **Anvil** provides a local Ethereum-compatible blockchain network with Chain ID 31337, allowing the complete application to be tested without using a public blockchain.
- **MetaMask** is used for wallet connection, account management, and transaction authorization.
- **React and JavaScript** are used to develop the frontend, providing the user interface through which administrators, companies, and employees interact with the decentralized application.

```
User@DESKTOP-EM24QK1 MINGW64 /
$ ls -lah ~/.foundry/bin
total 223M
drwxr-xr-x 1 User 197609    0 Sep  9 15:11 ./
drwxr-xr-x 1 User 197609    0 Sep  9 15:09 ../
-rwxr-xr-x 1 User 197609  40M Sep  9 15:09 anvil.exe*
-rwxr-xr-x 1 User 197609  57M Sep  9 15:09 cast.exe*
-rwxr-xr-x 1 User 197609  41M Sep  9 15:09 chisel.exe*
-rwxr-xr-x 1 User 197609  77M Sep  9 15:09 forge.exe*
-rwxr-xr-x 1 User 197609 3.8M Sep  9 15:08 foundryup*
-rwxr-xr-x 1 User 197609 6.4M Sep  9 15:09 solar.exe*

User@DESKTOP-EM24QK1 MINGW64 /
$ which forge
/c/Users/User/.foundry/bin/forge

User@DESKTOP-EM24QK1 MINGW64 /
$ forge --version
forge Version: 1.8.1
Commit SHA: 982849d3140c01fd3b72905759581a132df7aa98
Build Timestamp: 2026-08-28T17:48:03.905115800Z (1787939283)
Build Profile: dist

User@DESKTOP-EM24QK1 MINGW64 /
$ anvil --version
anvil Version: 1.8.1
Commit SHA: 982849d3140c01fd3b72905759581a132df7aa98
Build Timestamp: 2026-08-28T17:48:03.905115800Z (1787939283)
Build Profile: dist

User@DESKTOP-EM24QK1 MINGW64 /
$ cast --version
cast Version: 1.8.1
Commit SHA: 982849d3140c01fd3b72905759581a132df7aa98
Build Timestamp: 2026-08-28T17:48:03.905115800Z (1787939283)
Build Profile: dist

User@DESKTOP-EM24QK1 MINGW64 /
$
```

Figure 1 Tech Stacks Used for the Project

6. User Roles

StreamPay defines three primary user roles: Protocol Administrator, Company (Employer), and Employee.

- The **Protocol Administrator** is the account that deploys the smart contract and is responsible for managing the protocol fees accumulated from salary withdrawal transactions.
- The **Company (Employer)** can register employees, create salary streams, monitor outgoing streams, and cancel active streams when required.
- The **Employee** can view incoming salary streams, monitor the amount of salary that has vested over time, and withdraw the available earned salary. These role-based permissions ensure that each participant can perform only the operations associated with their responsibilities within the protocol.

7. Smart Contract Design

The StreamPay smart contract contains the complete financial logic of the protocol. Main components:

- Administrator management
- Company creation
- Employee registration
- Salary stream creation
- Time-based vesting calculation
- Employee withdrawal
- Protocol fee accounting
- Stream cancellation

The StreamPay smart contract contains the core financial and operational logic of the protocol. It manages administrator initialization, company creation, employee registration, salary stream creation, time-based vesting calculations, employee withdrawals, protocol fee accounting, and stream cancellation. Each salary stream maintains essential information such as the employer address, employee address, deposited ETH amount, stream start timestamp, streaming duration, withdrawn amount, and current stream status. By maintaining this information on-chain, the contract ensures that salary distribution and settlement are governed by predefined and transparent rules.

8. Administrator Initialization

During deployment, the account deploying the contract becomes the administrator.
The concept is:

```
constructor() {
    admin = msg.sender;
}
```

Because the deployment transaction is sent from the administrator wallet, the first Anvil account becomes the protocol administrator.

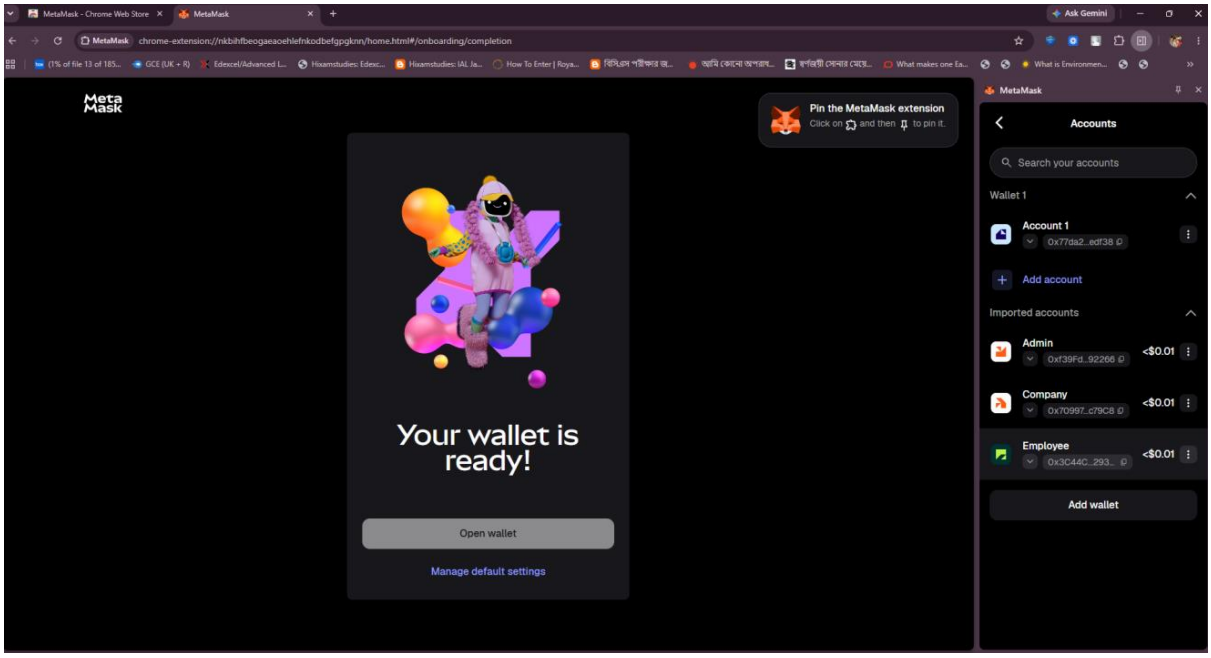

Figure 2 Accounts in Metamask

9. Salary Stream Creation

When a Company creates a salary stream, it provides the employee's blockchain wallet address, the salary amount in ETH, and the desired streaming duration. The smart contract validates the request and records the stream information on the blockchain while locking the deposited ETH within the contract. Once the stream is created, the employee becomes entitled to receive the salary progressively according to the elapsed time. Therefore, the overall process follows the sequence of the employer initiating the stream, the smart contract locking and managing the deposited funds, and the employee gradually earning access to the salary as the stream progresses.

Workflow:

Employer → Create Stream → Smart Contract → Employee Receives Stream.

10. Vesting Calculation

StreamPay uses linear vesting.

Formula: Unlocked Amount = (Total Salary × Time Passed) / Total Duration

Example:

A 1 ETH stream with 120 seconds duration:

After 60 seconds:

Unlocked Amount = (1 × 60) / 120

Unlocked Amount = 0.5 ETH

This ensures employees receive salary proportionally according to completed time.

11. Withdrawal Mechanism

The withdrawal mechanism allows employees to claim the portion of their salary that has already vested. When an employee submits a withdrawal request, the smart contract first calculates the total

amount that should have been unlocked according to the elapsed streaming time. The amount previously withdrawn from the stream is then deducted to determine the employee's currently claimable balance. The contract subsequently calculates the applicable protocol fee and transfers the remaining amount to the employee. The corresponding fee is recorded in the protocol's internal fee balance so that it can later be claimed by the Protocol Administrator. This mechanism prevents employees from withdrawing the same vested funds more than once.

Process:
1. Employee requests withdrawal.
2. Contract calculates available vested amount.
3. Previously withdrawn amount is deducted.
4. Protocol fee is calculated.
5. Employee receives the remaining amount.
6. Fee is stored for administrator collection.

12. Stream Cancellation

Cancellation allows a stream to end before completion.

The settlement process:
1. Calculate employee vested salary.
2. Transfer earned amount to employee.
3. Calculate employer refund.
4. Close the stream.

Stream cancellation allows a Company to terminate an active salary stream before its scheduled completion. When a stream is cancelled, the smart contract first calculates the amount of salary that has vested up to the cancellation time. The employee's earned amount is settled according to the protocol's withdrawal and fee rules, while the remaining unvested ETH is returned to the employer. After settlement, the stream is marked as closed, preventing further withdrawals or cancellation operations. This approach ensures that the employee receives compensation for the time already worked while the employer recovers the portion of the salary that had not yet vested.

13. Frontend Implementation

The frontend provides separate dashboards.

Admin Dashboard:
• Protocol fee balance
• Network information
• Claim fee option

Employer Dashboard:
• Employee registration
• Salary stream creation
• Outgoing stream management

Employee Dashboard:
• Incoming streams

• Vesting progress
• Withdrawal option

The StreamPay frontend provides separate interfaces for the Protocol Administrator, Company, and Employee. The **Admin Dashboard** displays protocol-related information such as the accumulated fee balance, network information, and the option to claim available protocol fees. The **Employer Dashboard** provides functionality for registering employees, creating salary streams, monitoring outgoing streams, and managing active streams. The **Employee Dashboard** allows employees to view their incoming salary streams, monitor vesting progress, and withdraw their currently available salary. The role-specific interfaces ensure that users are presented with the functions relevant to their permissions.

14. Checkpoint 1 - Environment Setup

Implemented requirements:
• Foundry installed.
• Anvil blockchain running.
• MetaMask connected.
• Chain ID configured as 31337.
• Development accounts imported.

The development environment was configured using Foundry, Anvil, and MetaMask. Foundry was used as the development and testing framework for the Solidity smart contract, while Anvil provided the local Ethereum-compatible blockchain network. The local network was configured with Chain ID 31337 as required by the project specification.

MetaMask was connected to the local Anvil network, and the pre-funded Anvil accounts were imported for testing. Account # 0 was reserved as the Protocol Administrator and was used to deploy the StreamPay smart contract.

The following Figure 3 shows the successfully configured development environment, including the tools required to build and run the StreamPay application.

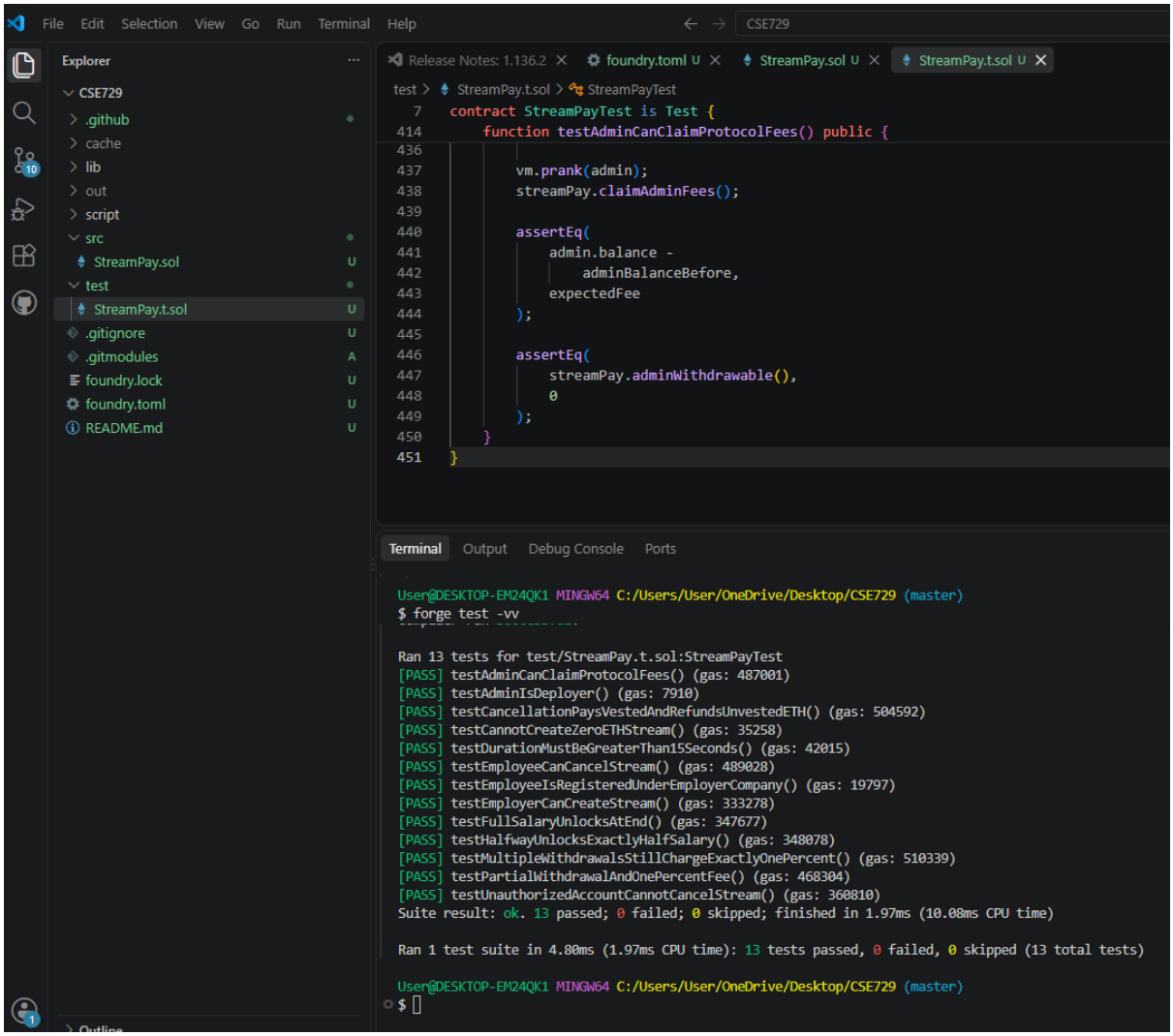

Figure 3 Successful Setups

The following Figure 4 shows the Anvil local blockchain running successfully and providing the local Ethereum network required by the DApp.

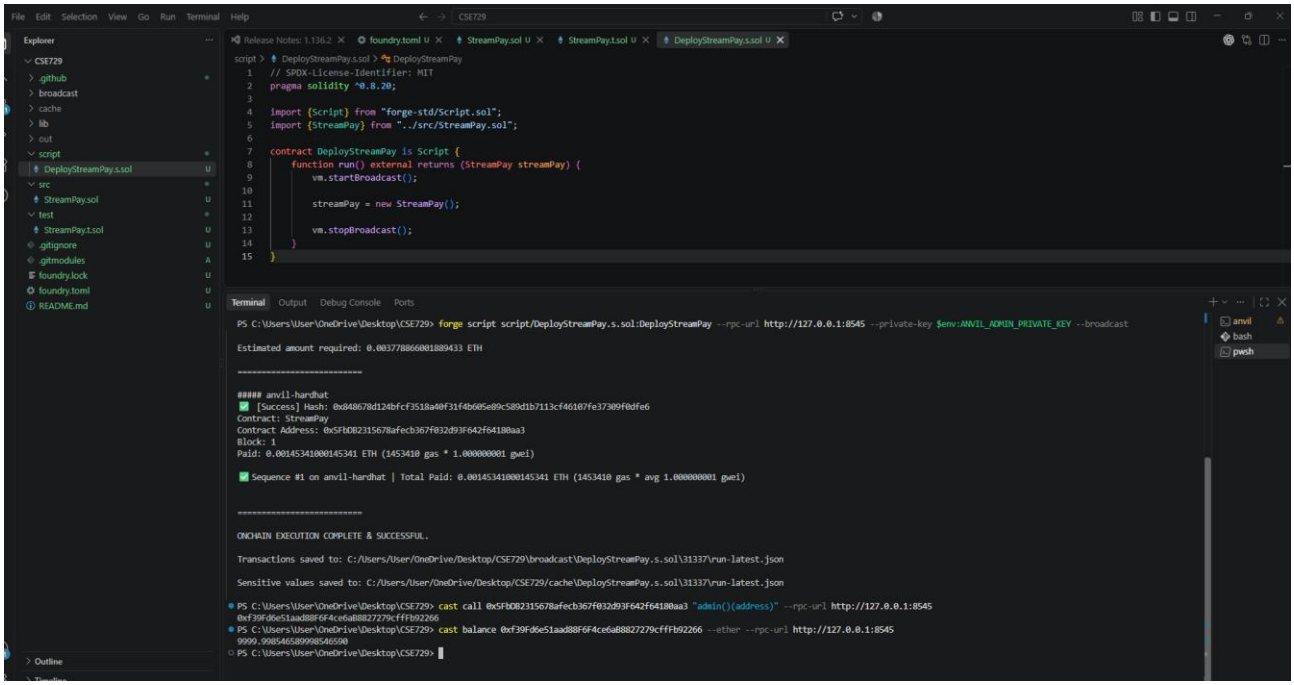

Figure 4 Anvil Running

The following Figure 5 shows the Anvil-funded accounts imported into MetaMask for testing. Account # 0 is used as the Protocol Administrator.

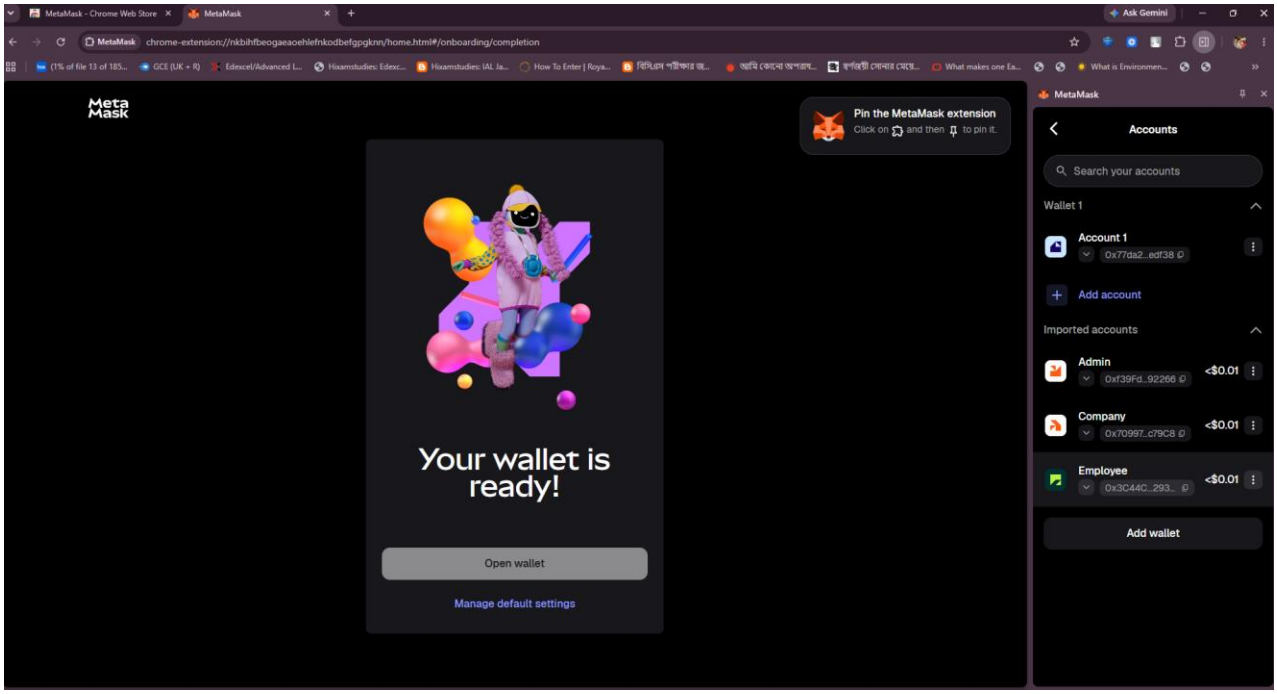

Figure 5 Metamask Accounts Created

15. Checkpoint 2 - Smart Contract Deployment

The StreamPay contract was deployed using Foundry scripts.

Implementation included:

• Deployment command.
• Successful deployment output.
• Contract address.
• Administrator verification.

The StreamPay smart contract was successfully compiled and deployed to the local Anvil blockchain using Foundry. The deployment was performed from Anvil Account # 0, which is designated as the Protocol Administrator.

The deployed contract stores the administrator address during construction. Therefore, the account used for deployment becomes the protocol administrator and is authorized to manage accumulated protocol fees.

The successful deployment and subsequent frontend connection confirm that the smart contract and decentralized client layer were correctly integrated.

The following Figure 6 shows the successful deployment of the StreamPay smart contract and the resulting contract address on the local Anvil network.

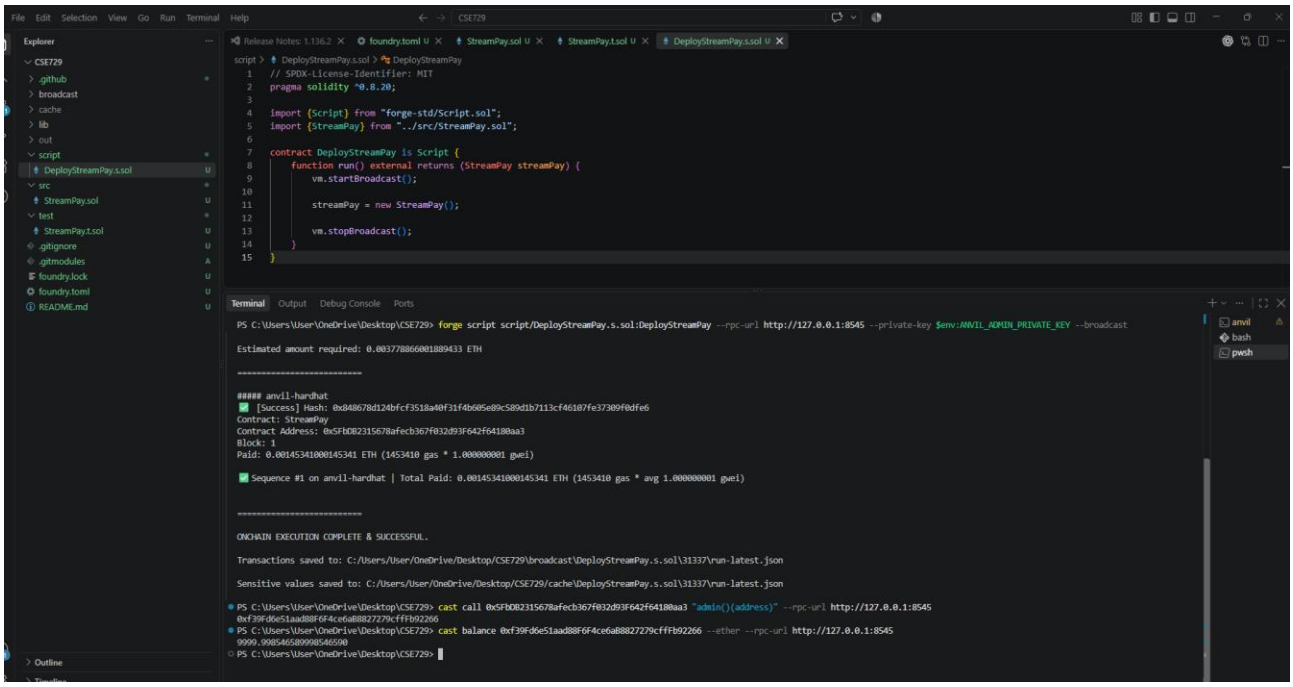

Figure 6 Contract Address

The following Figure 7 shows the initial Admin interface before a wallet is connected to the DApp.

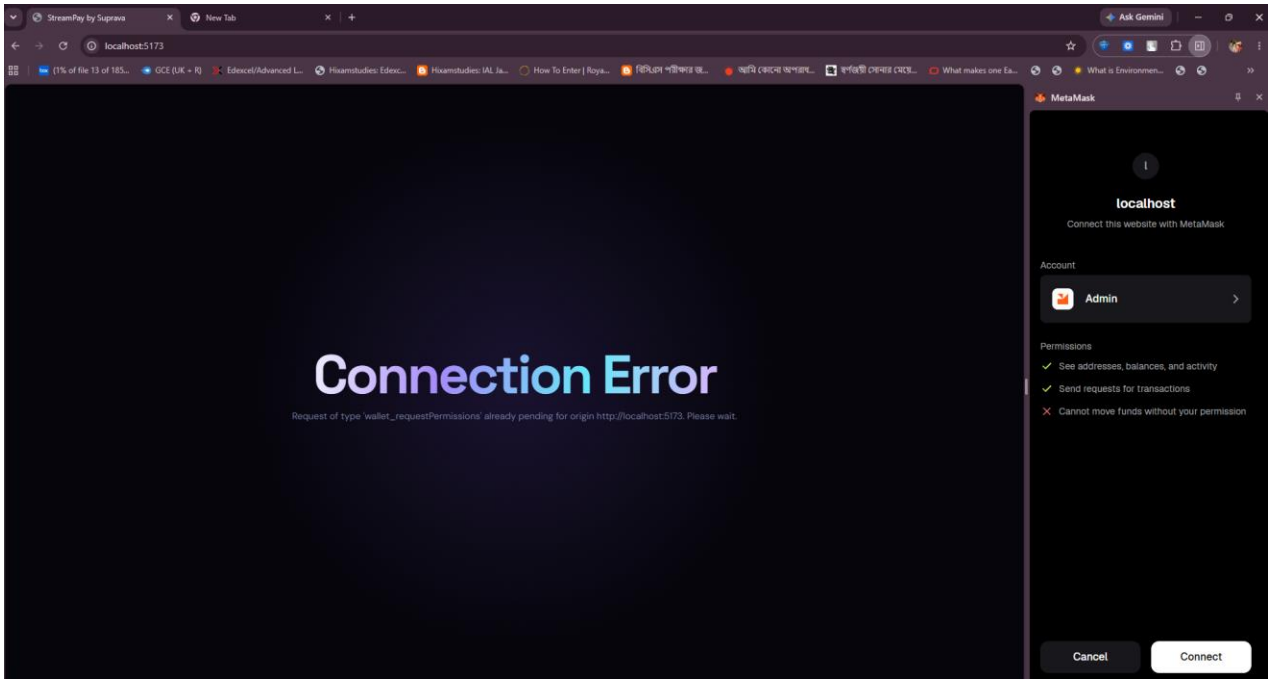

Figure 7 Admin Frontend before Connecting

After connecting the administrator wallet, the DApp identifies the account as the Protocol Administrator and displays the corresponding Admin dashboard.

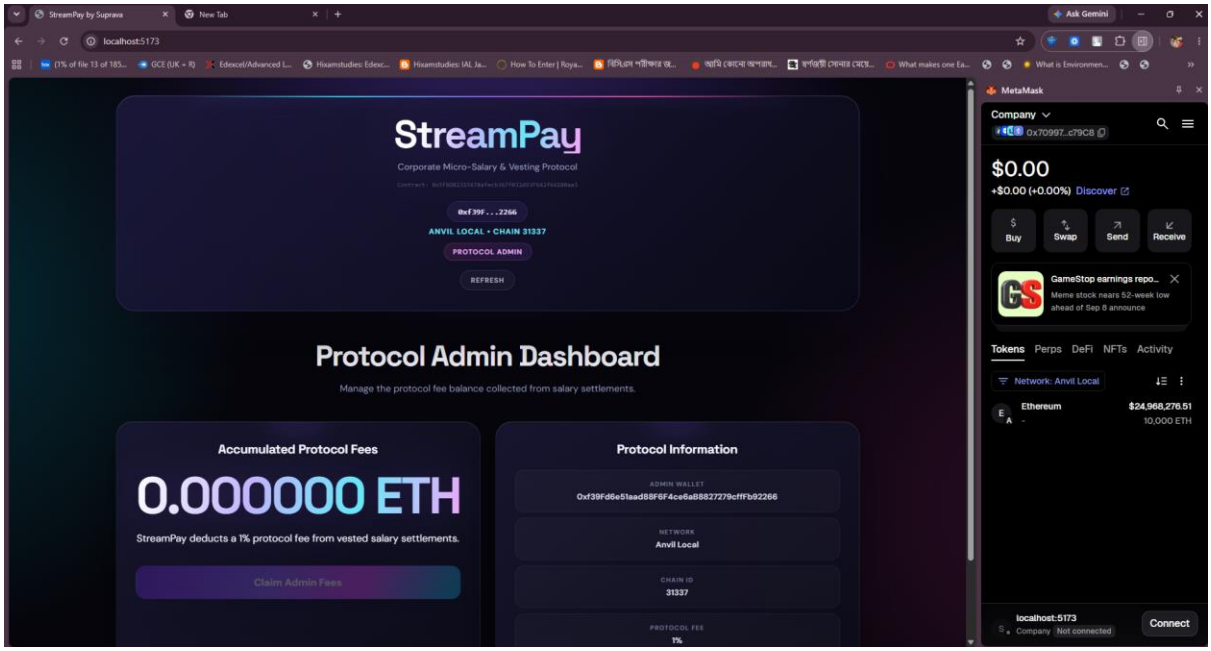

Figure 8 Admin Frontend After Connecting

After connecting an employer account, the DApp detects the Company role and displays the employer-specific dashboard and available stream management functions as Figure 9 shows.

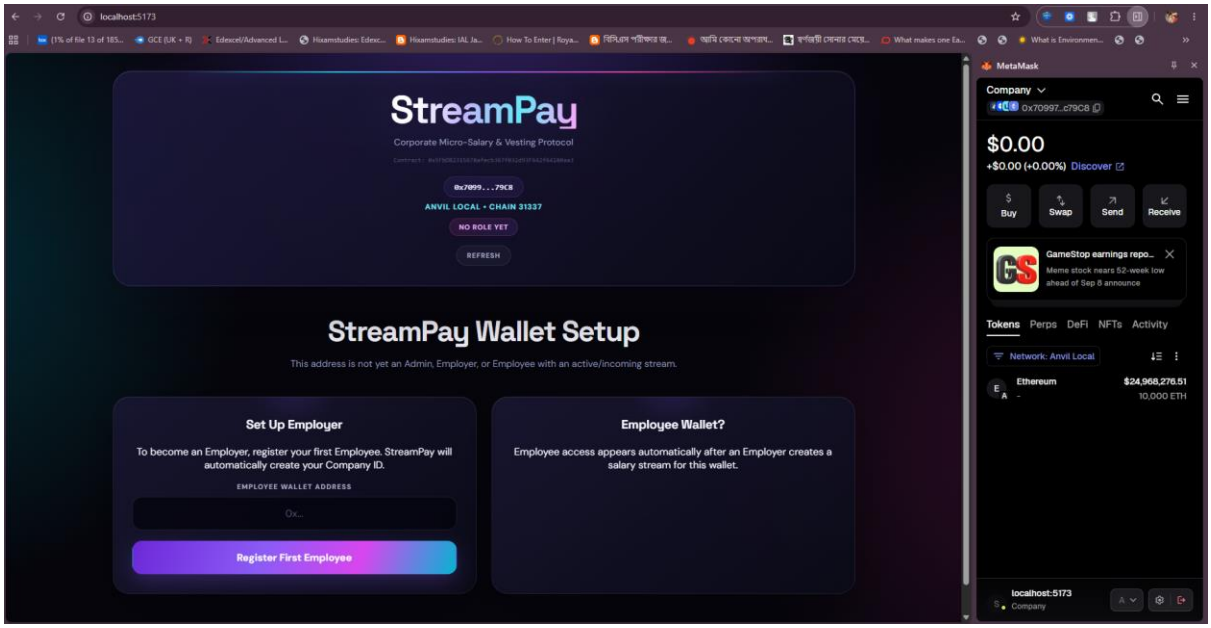

Figure 9 Company Frontend After Connecting

After connecting an employee account, the DApp detects the Employee role and displays the incoming salary stream interface in the figure-10.

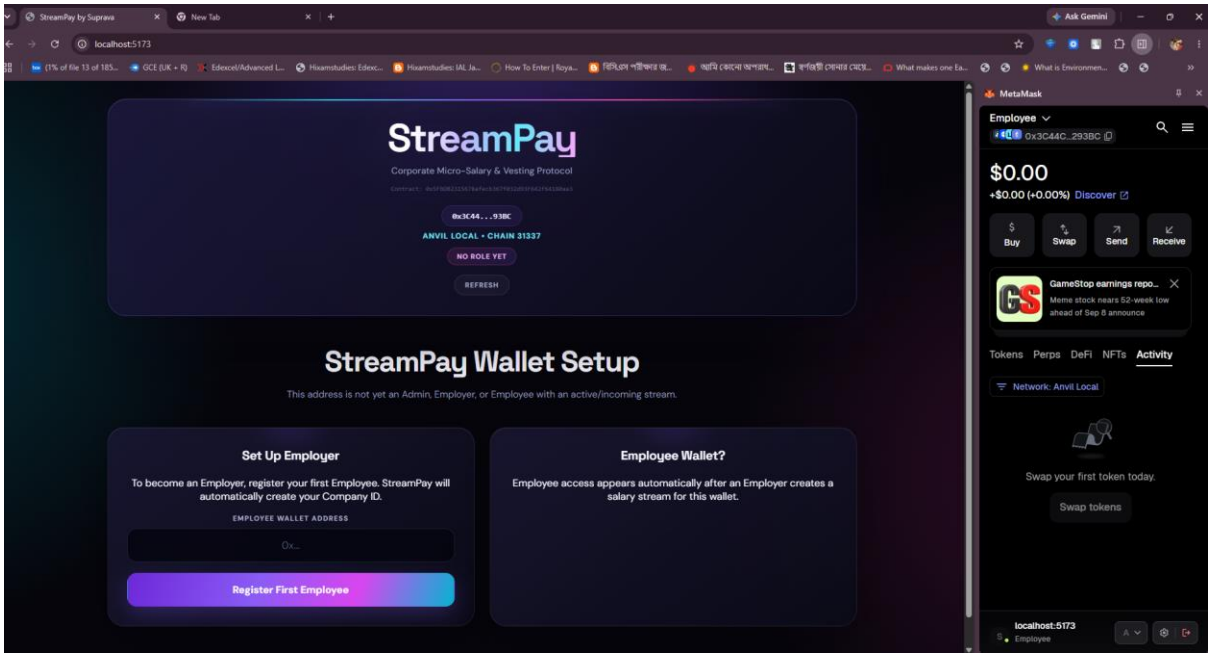

Figure 10 Employee Frontend After Connecting

16. Checkpoint 3 - Real-Time Frontend

The frontend provides live updates of salary streams.

Features:

• Role detection.

• Employee balance display.

• Real-time vesting calculation.
• Dashboard updates.

The StreamPay DApp implements a role-based React interface that dynamically changes according to the connected MetaMask wallet. After connecting to the local Anvil network, the application identifies the active wallet address and displays the appropriate dashboard for the Protocol Administrator, Company, or Employee.

The Employee dashboard provides the real-time salary streaming interface. Instead of querying the blockchain every second, the application initially retrieves the required stream state from the smart contract, including the start timestamp, stream duration, total deposited amount, and total withdrawn amount. The frontend then uses a JavaScript timer running at one-second intervals to calculate the currently unlocked amount locally.

The calculation follows the same integer-based vesting formula used by the smart contract:

Unlocked Amount = (Total Deposit × Time Elapsed) / Total Duration

The claimable amount is then determined by subtracting the amount that has already been withdrawn. This allows the displayed balance to increase continuously without sending repeated RPC requests to the blockchain.

The following figure-11 shows the Company dashboard after wallet connection, providing the interface for managing employees and outgoing salary streams.

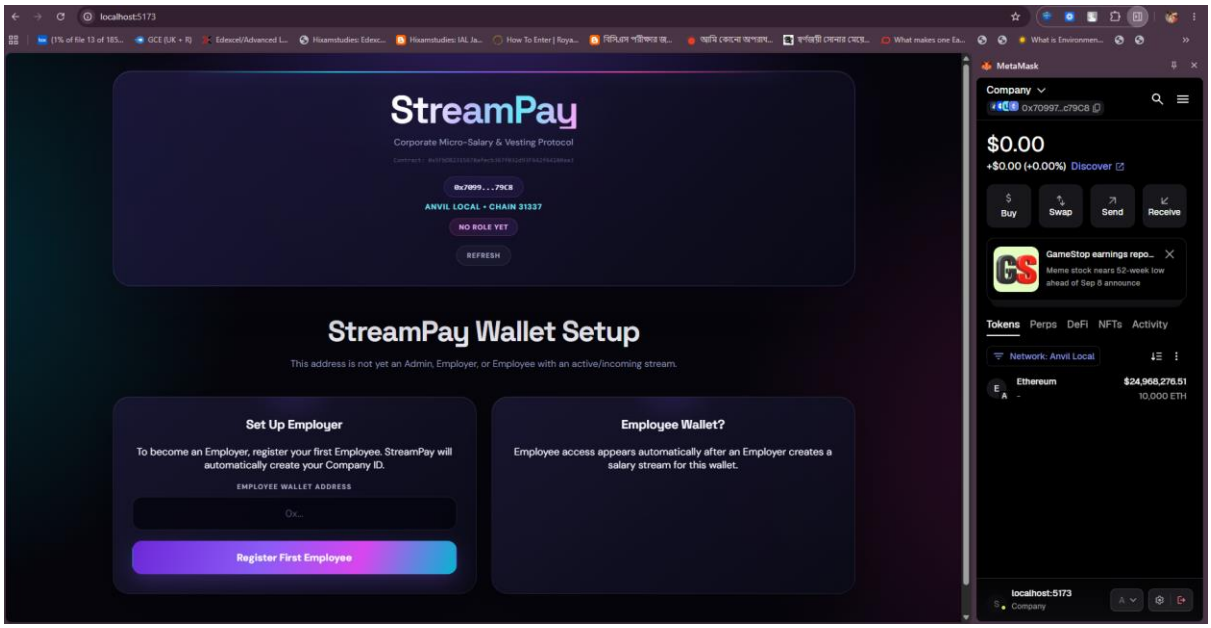

Figure 11 Company Frontend After Connecting

The following figure-12 shows the Employee dashboard, where the employee can view the incoming stream and monitor the currently claimable salary.

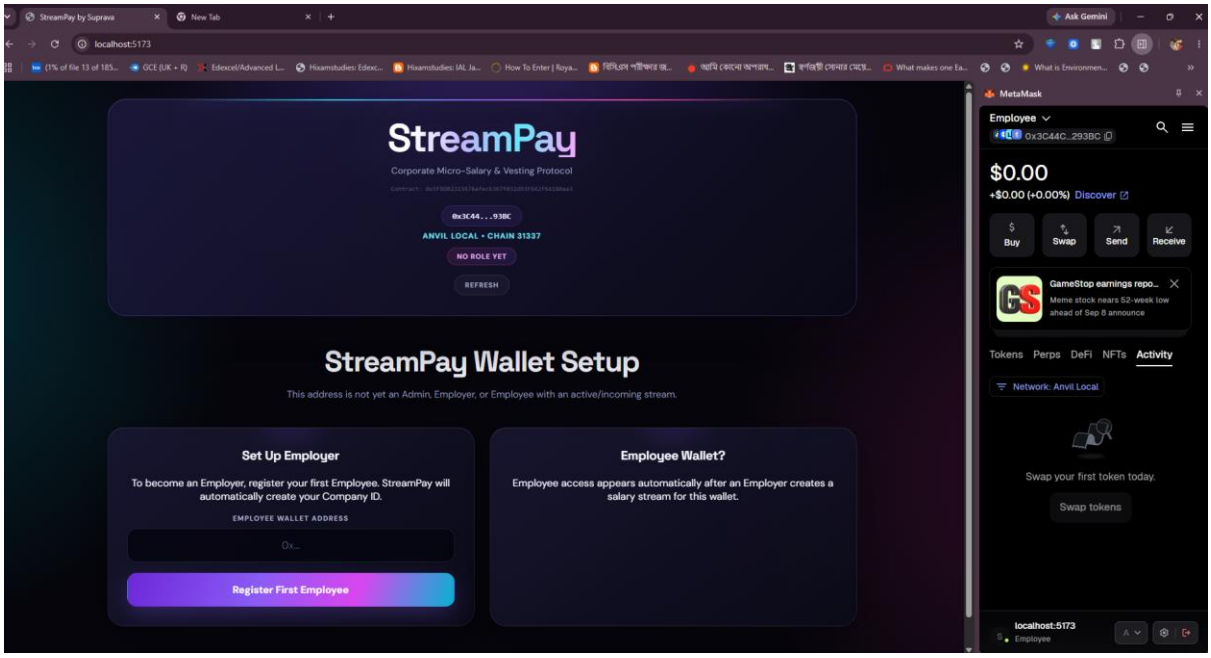

Figure 12 Employee Frontend After Connecting

17. Checkpoint 4 - Streaming Execution Flow

Complete execution flow:

1. Employer registers employee.
2. Employer creates salary stream.
3. Employee receives stream.
4. Salary starts vesting.
5. Employee withdraws funds.

The fourth checkpoint demonstrates the complete StreamPay workflow, beginning with employee registration and continuing through stream creation, continuous salary vesting, withdrawal, protocol fee collection, and partial withdrawal. The transactions were executed through MetaMask and confirmed on the local Anvil blockchain.

The following Figure 13 shows the Company initiates employee registration by providing the employee's blockchain wallet address.

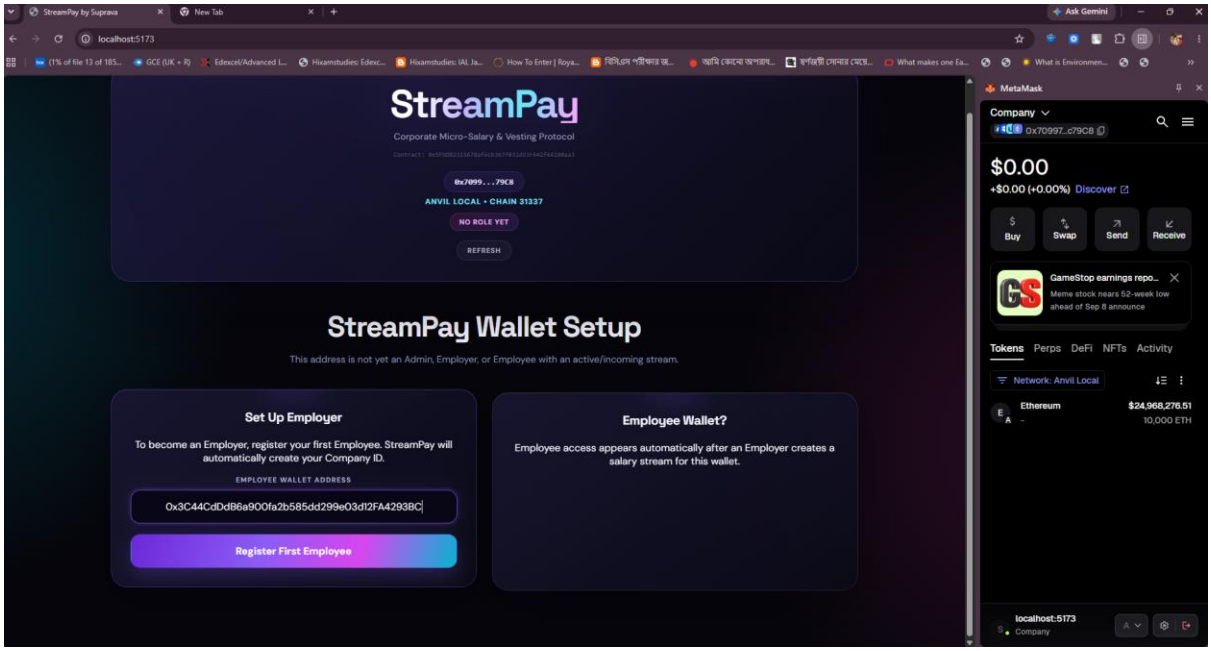

Figure 13 Company Registers an Employee using Wallet Address

Figure 14 demonstrates the MetaMask requests confirmation from the Company before submitting the employee registration transaction to the blockchain.

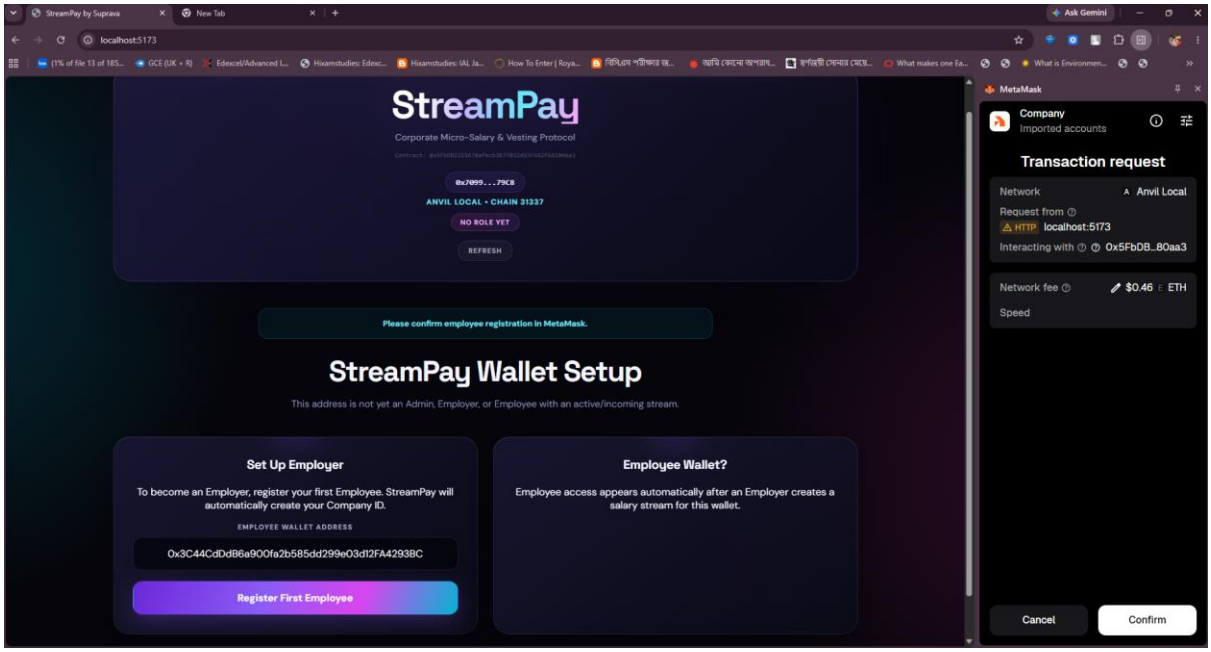

Figure 14 Metamask asks for Confirmation

The following Figure 15 shows the successful transaction confirms that the employee has been registered under the Company.

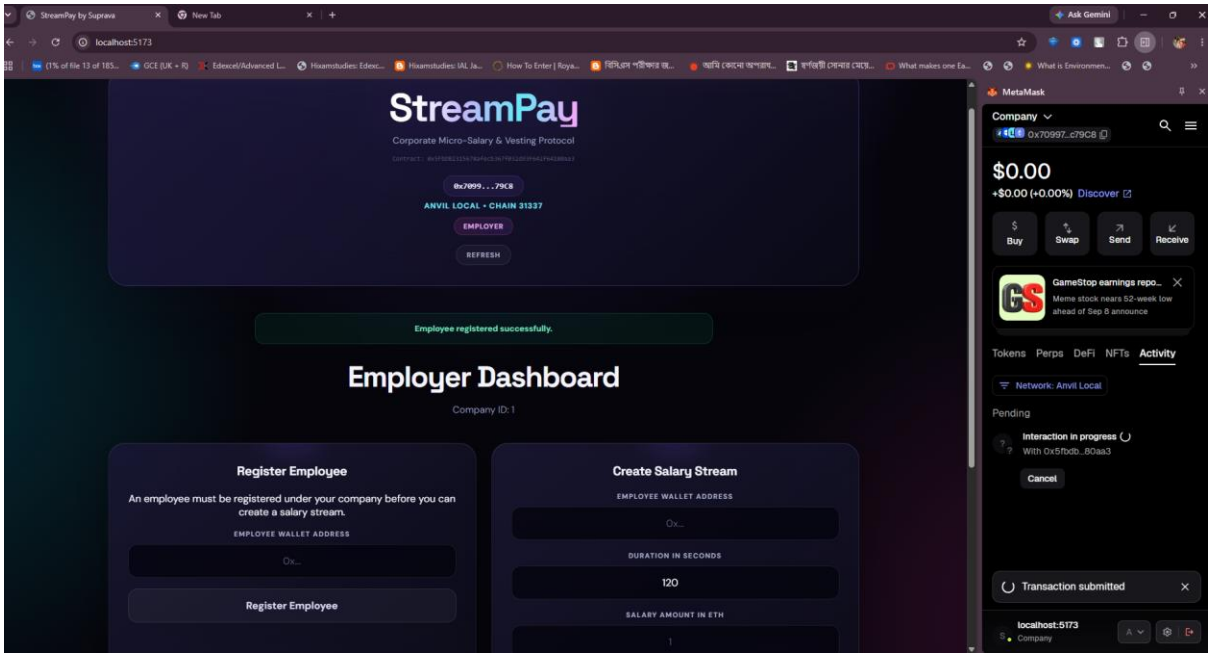

Figure 15 Employee Successfully Registered

The following figure-16 shows the Company enters the employee wallet address, salary amount, and streaming duration to create a new salary stream.

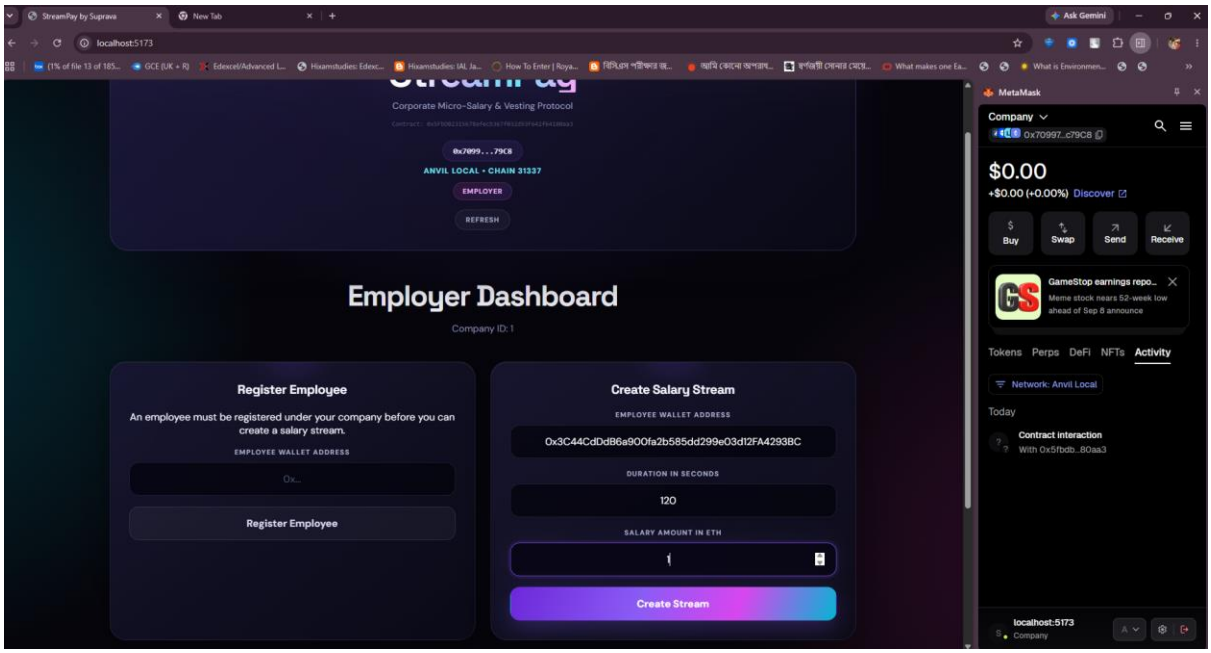

Figure 16 Salary Stream being created by the Company by providing inputs

The following Figure 17 shows the successful transaction confirms that the salary stream has been created and the deposited ETH is now locked in the smart contract.

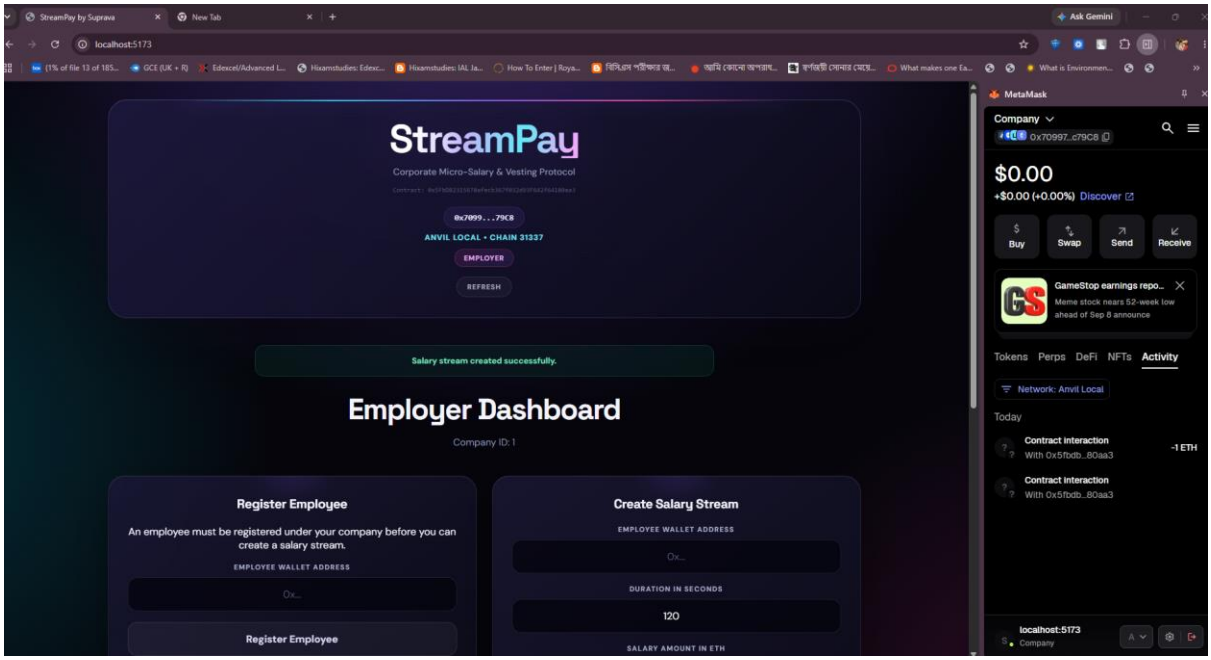

Figure 17 Salary Stream Successfully Created by the Company

The Employee dashboard displays the claimable amount after the stream has started and a portion of the salary has vested in the Figure 18.

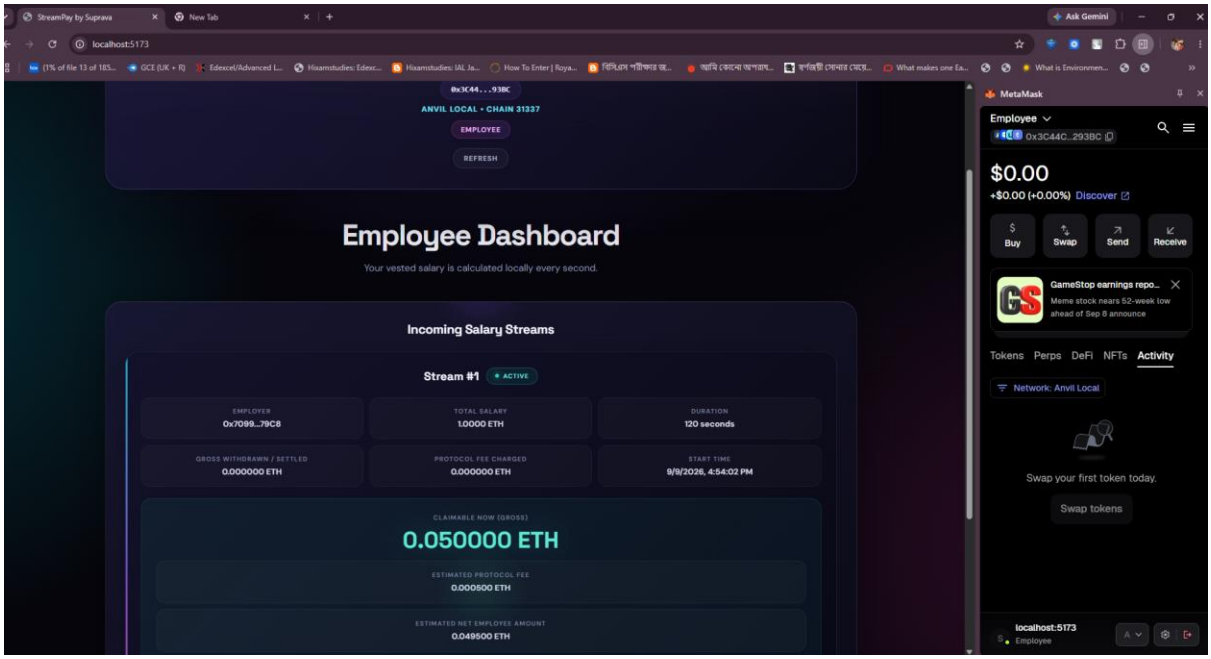

Figure 18 Salary being Received the Employee in Realtime (State 1)

After additional time has elapsed, the claimable salary increases according to the linear vesting calculation in the Figure 19.

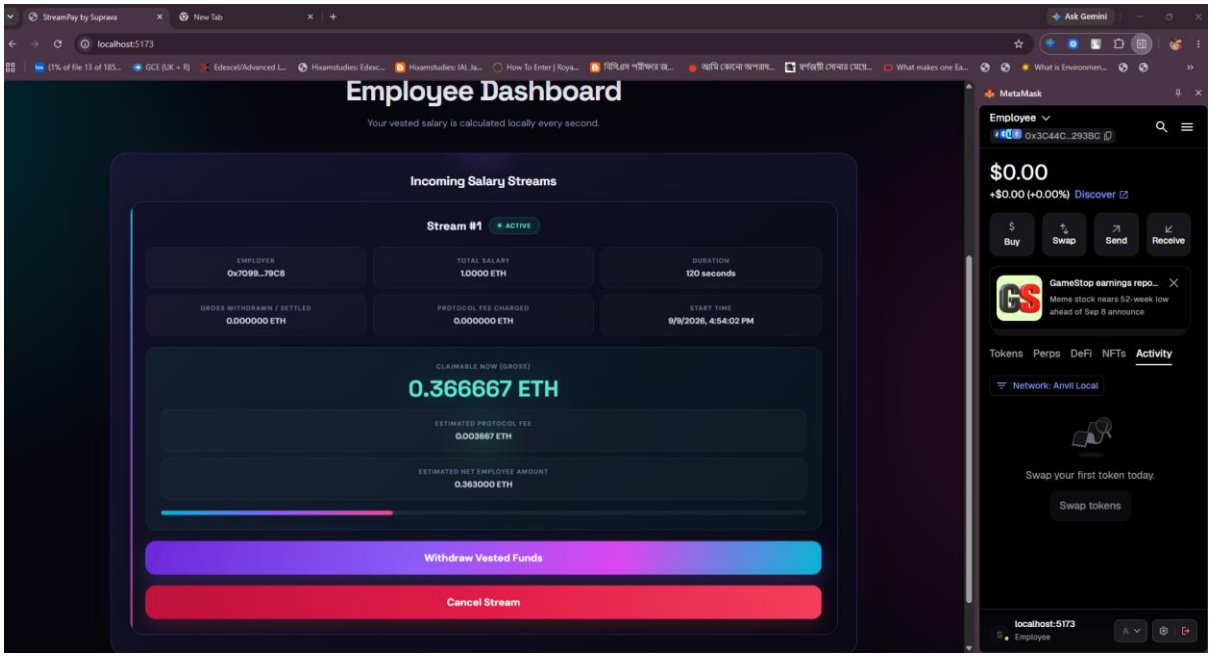

Figure 19 Salary being Received the Employee in Realtime (State 2)

The Employee successfully receives the vested salary after initiating the withdrawal operation in the following Figure 20.

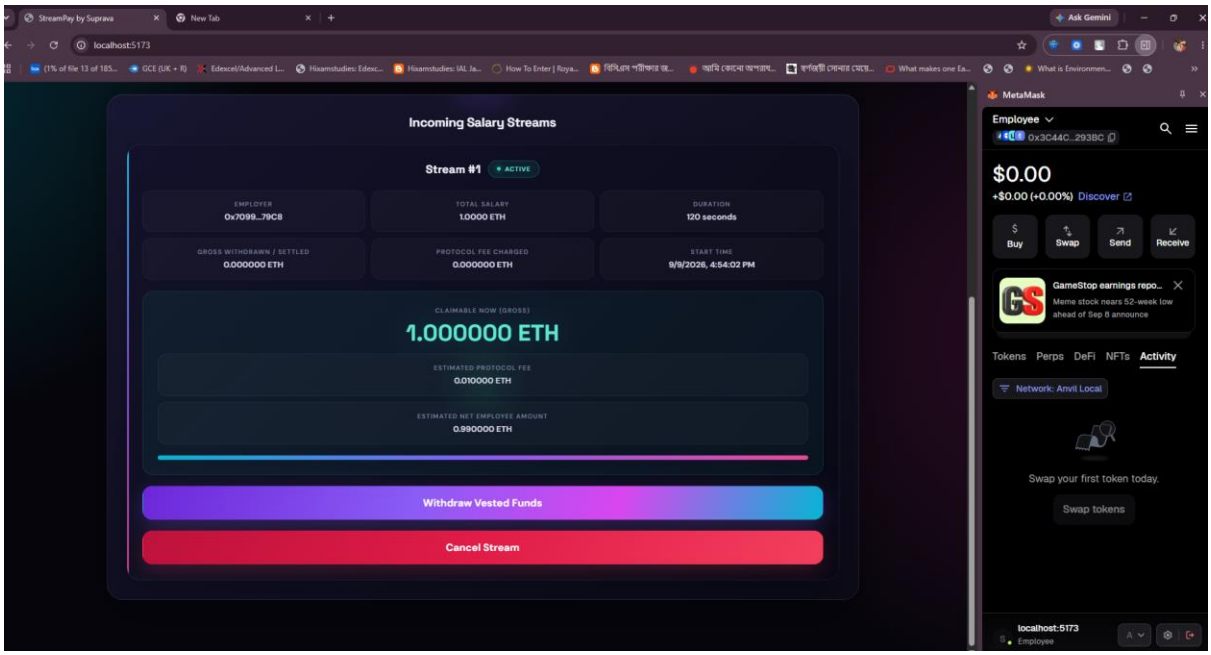

Figure 20 Salary Successfully Received by the Employee

The following Figure 21 shows the Employee initiates a withdrawal request to claim the currently available vested amount.

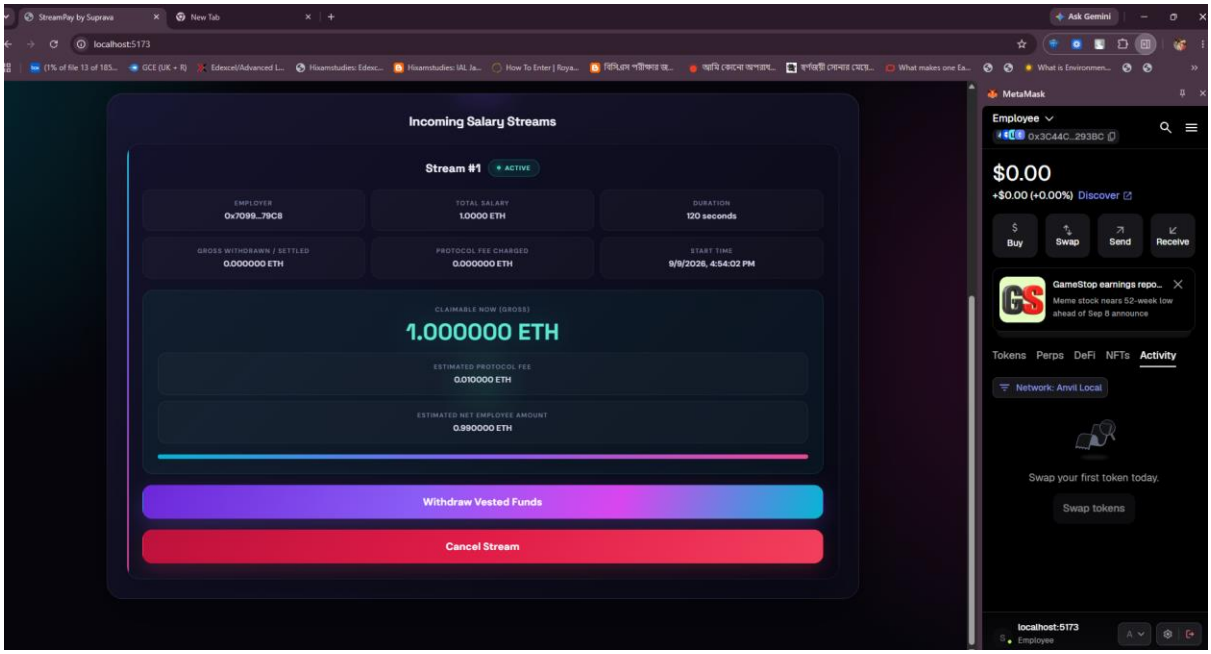

Figure 21 Withdrawal Request by the Employee

The successful transaction confirms that the vested amount has been transferred to the Employee and recorded as withdrawn by the smart contract.

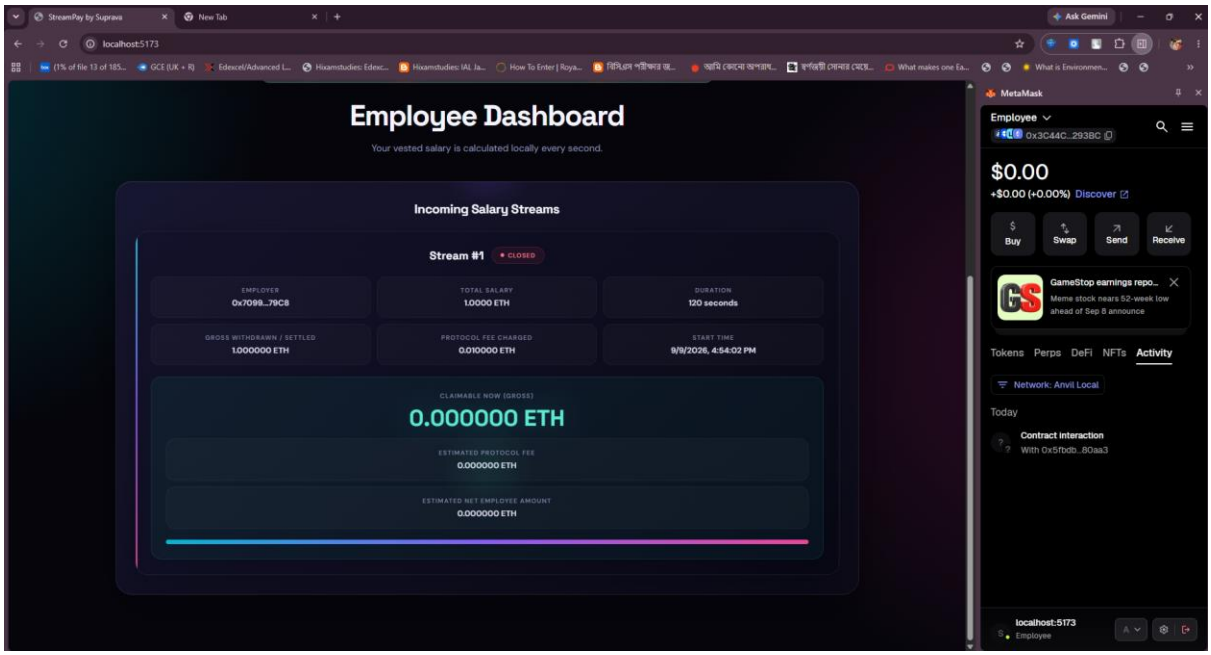

Figure 22 Successful Withdrawal by the Employee

In the following Figure 23 shows the withdrawal, the 1% protocol fee is added to the administrator's internal withdrawable balance.

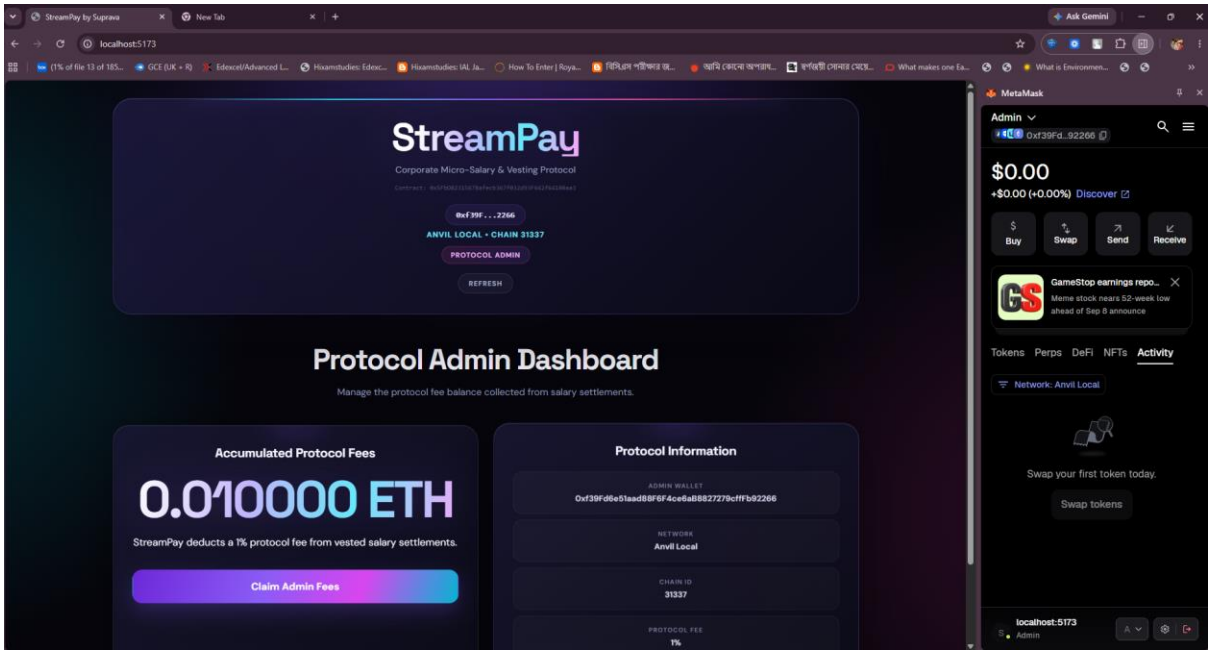

Figure 23 Admin Fee getting Added

In the following Figure 24, the Protocol Administrator initiates a transaction to claim the accumulated protocol fees.

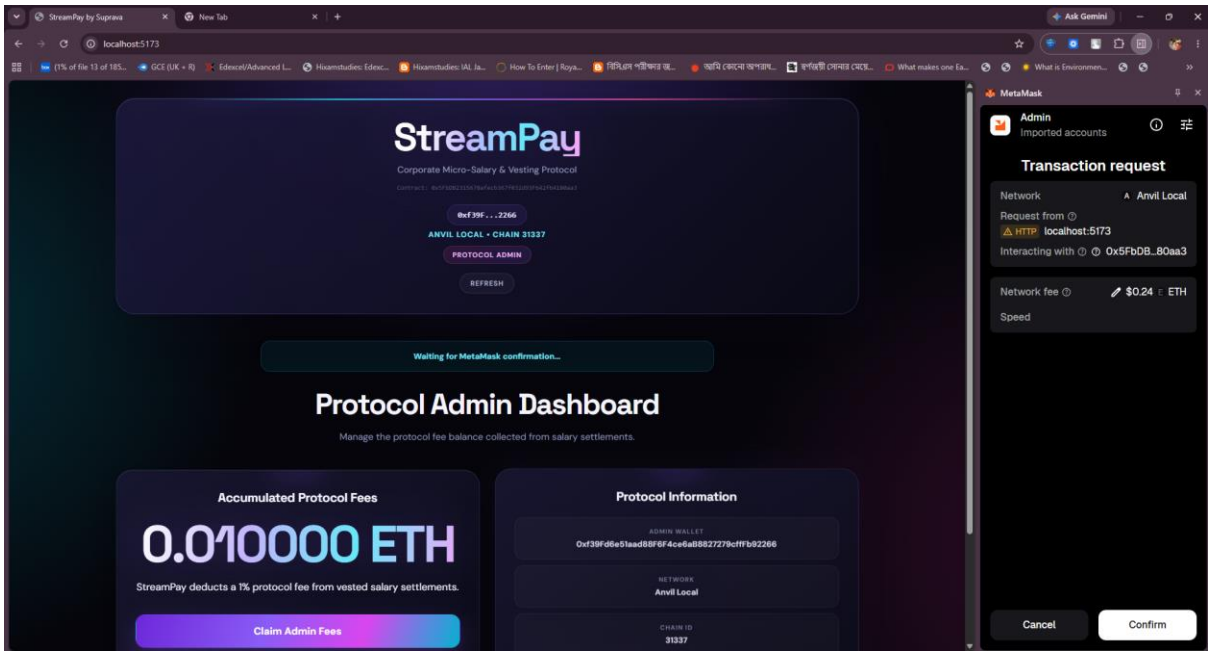

Figure 24 Admin Fee withdrawal Request by the Admin

The successful transaction confirms that the accumulated protocol fee has been transferred to the administrator in the following Figure 25.

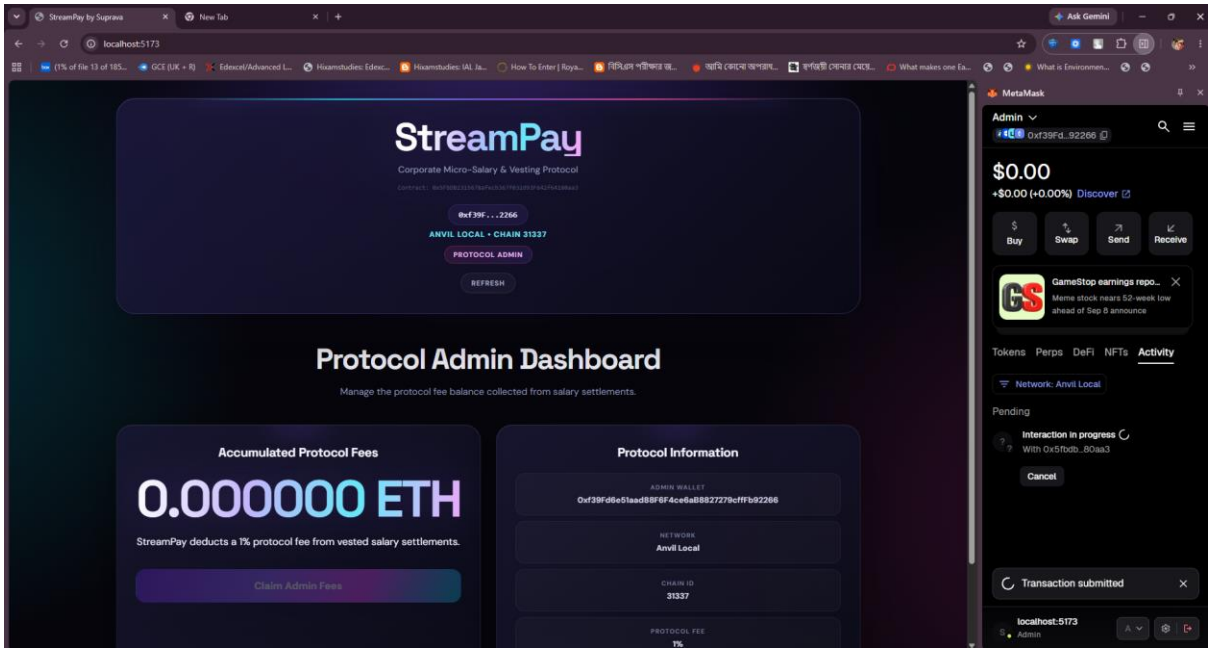

Figure 25 Successful Withdrawal of the Admin Fee

Following Figure 26 shows a second salary stream is created to demonstrate partial withdrawal while the stream remains active.

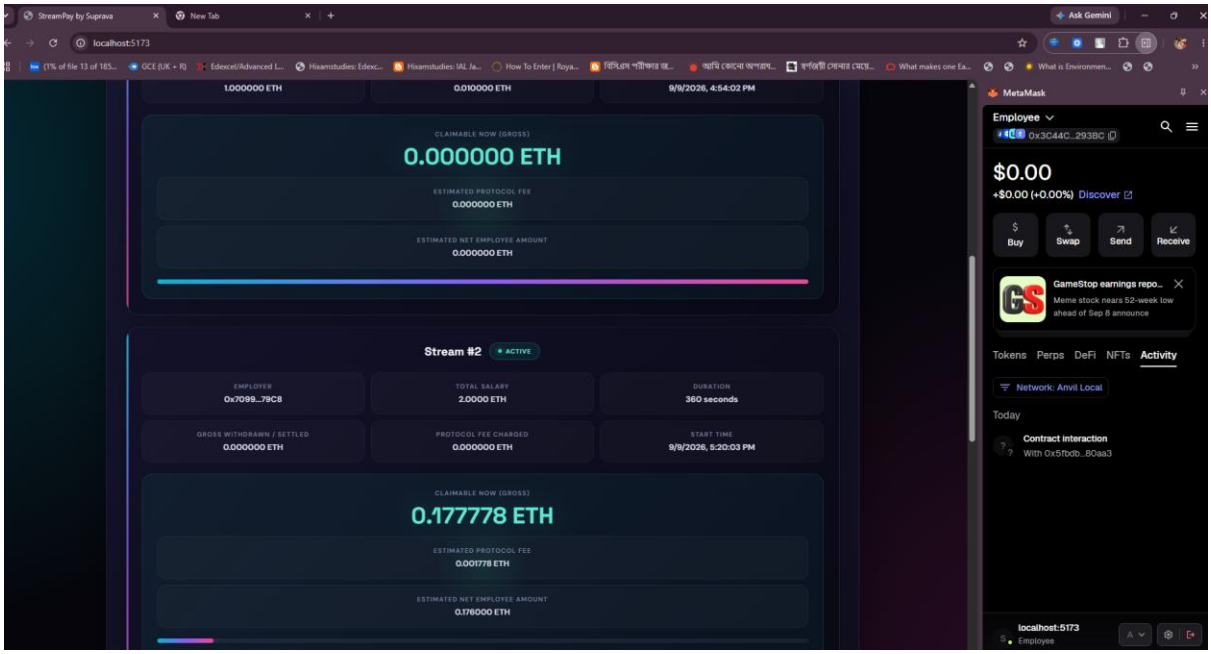

Figure 26 Stream 2 Created by the Company

The Employee initiates a partial withdrawal while Stream 2 is still active and additional salary continues to vest.

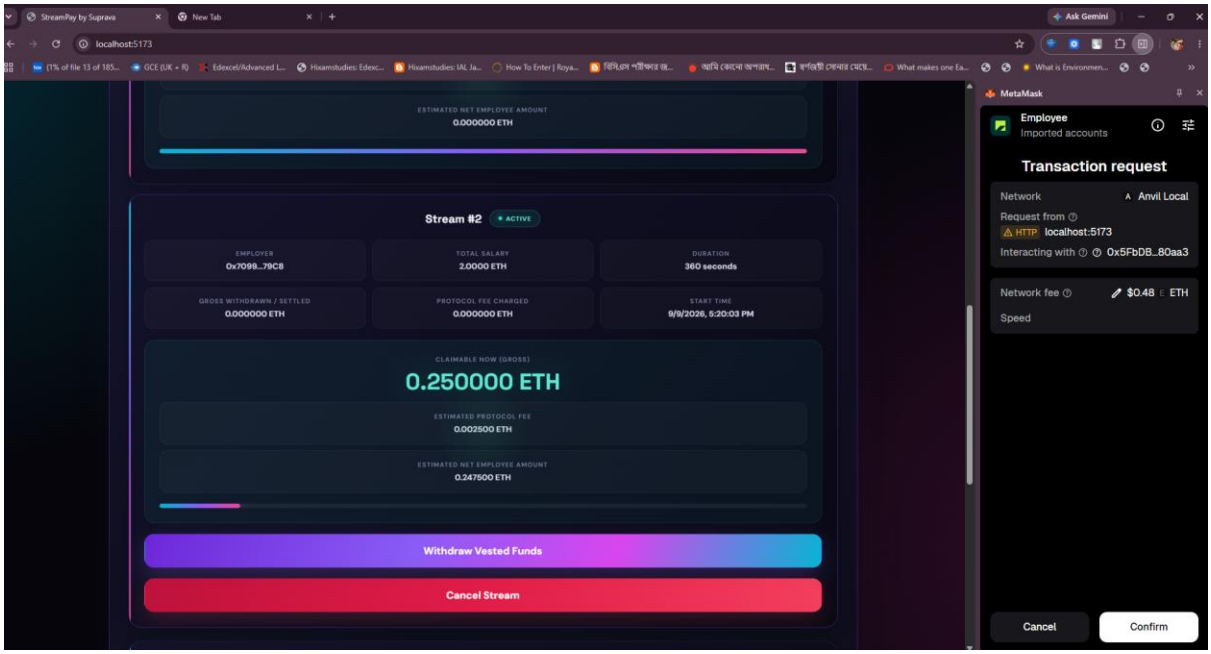

Figure 27 Partial Withdrawal Request by the Employee

The successful transaction confirms that the Employee received the newly vested portion partially.

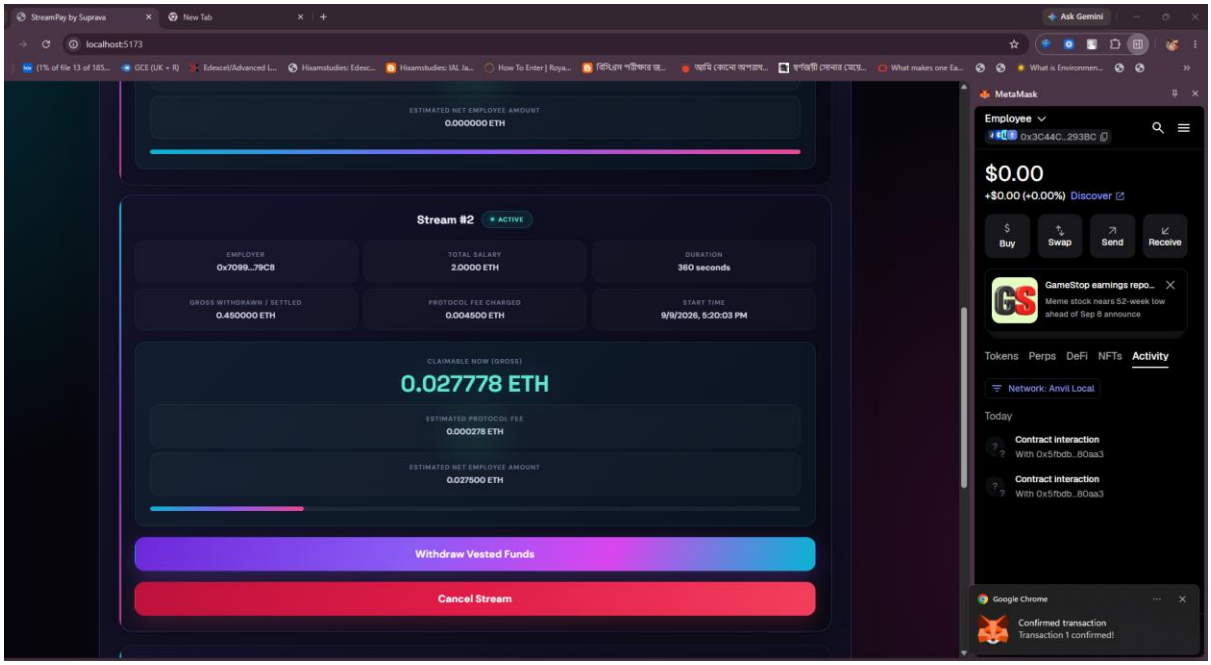

Figure 28 Successful Partial Withdrawal by the Employee

18. Bonus Checkpoint - Live Event Auto Sync

Stream status changes are reflected between users. Example:

Before cancellation:

ACTIVE STREAM

After cancellation:

CLOSED STREAM

The bonus checkpoint demonstrates live synchronization between the Employer and Employee interfaces. The Employer cancels an active stream from one browser window while the Employee monitors the same stream from another window. The application listens for the relevant blockchain state change and updates the affected interface without requiring a manual page reload.

The following Figure 29 shows that the Company initiates cancellation of Stream 2 while the stream is still active.

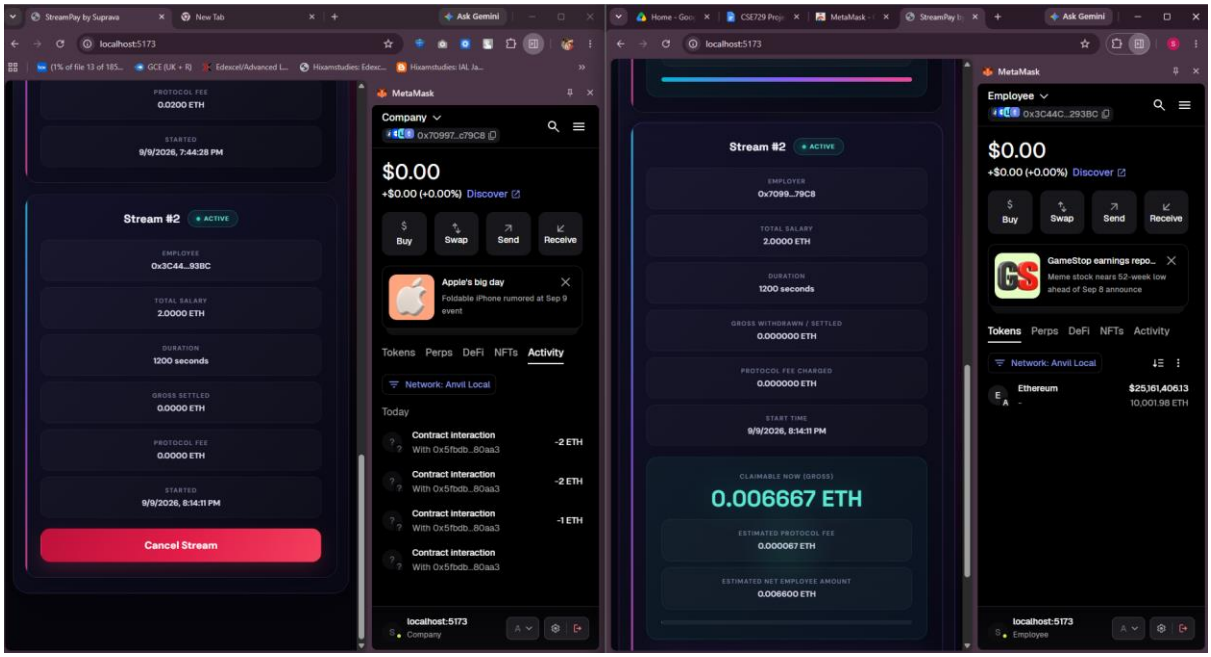

Figure 29 Stream 2 Cancellation Request by the Company

The application displays the first confirmation stage for the stream cancellation transactioncin the following Figure 30.

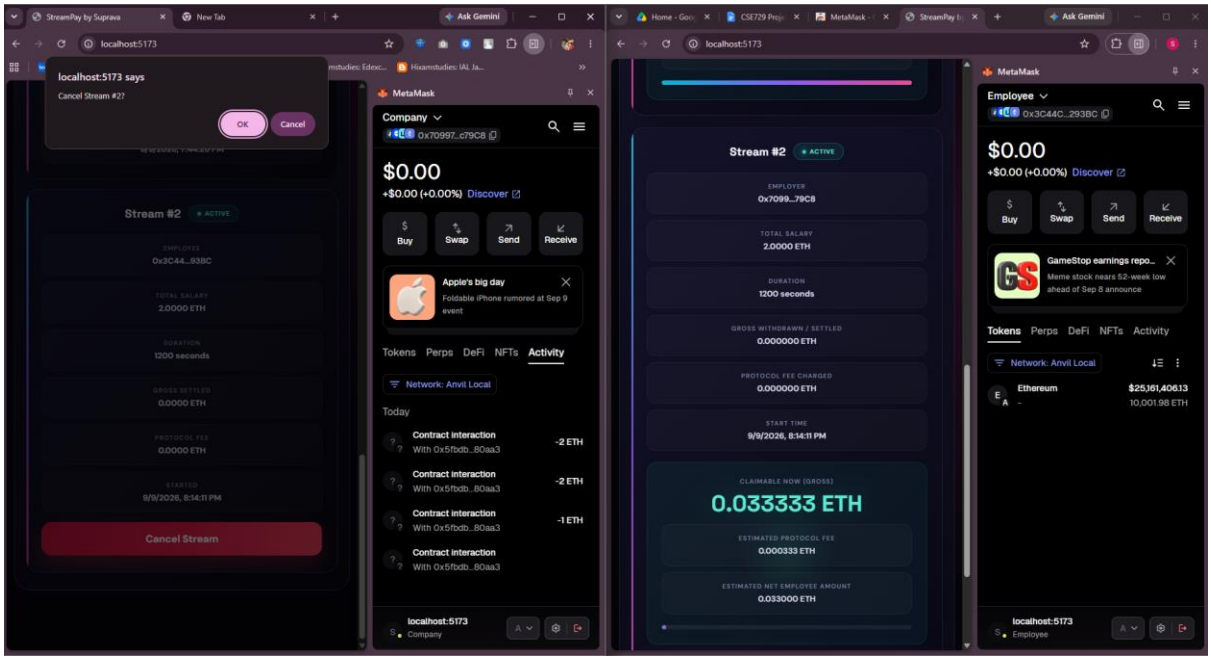

Figure 30 Steam 2 Cancellation Confirmation 1 by the Company

In Figure 31, MetaMask requests authorization from the Company before submitting the cancellation transaction.

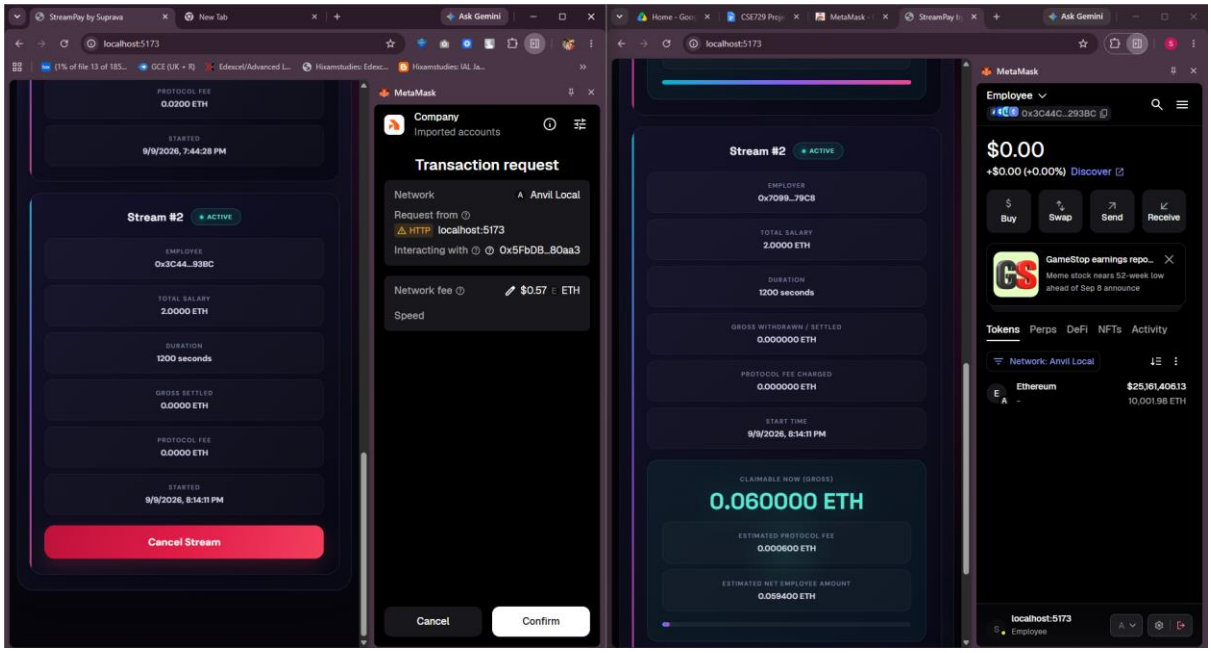

Figure 31 Steam 2 Cancellation Confirmation 2 in the Metamask by the Company

In Figure 32, the successful transaction confirms that Stream 2 has been closed.

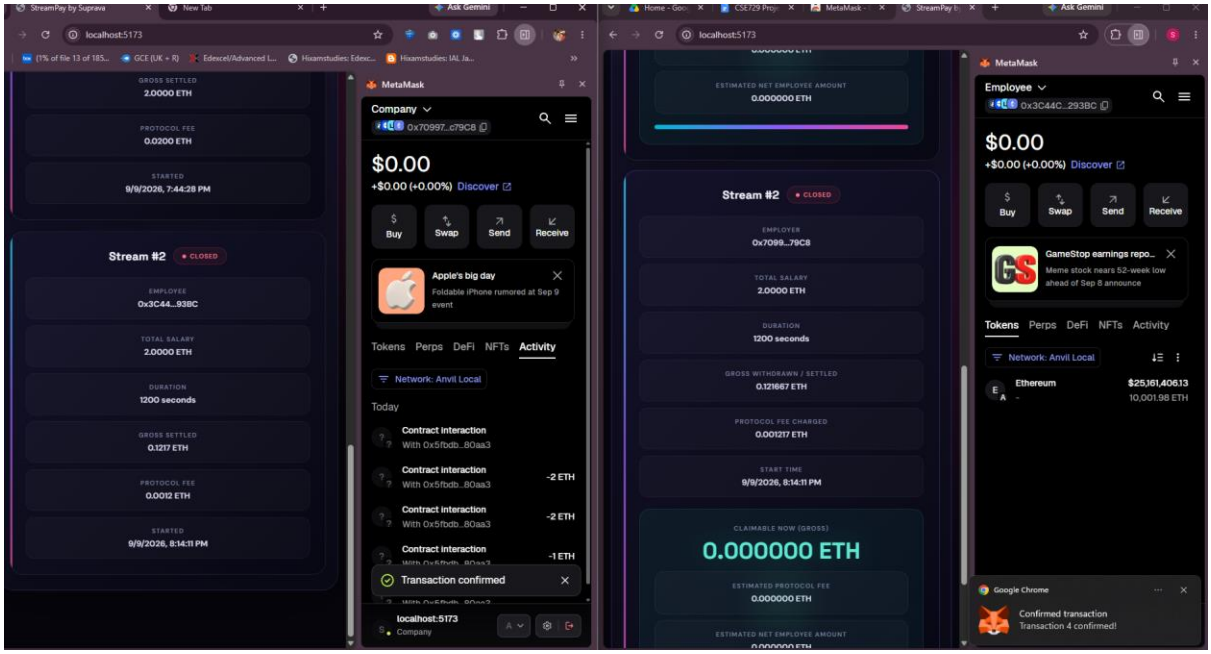

Figure 32 Steam Successfully Cancelled by the Employee

The following Figure 33 demonstrates the live synchronization between the Employer and Employee interfaces. After the Employer cancels Stream 2, the Employee interface immediately reflects the closed stream state without requiring a page reload.

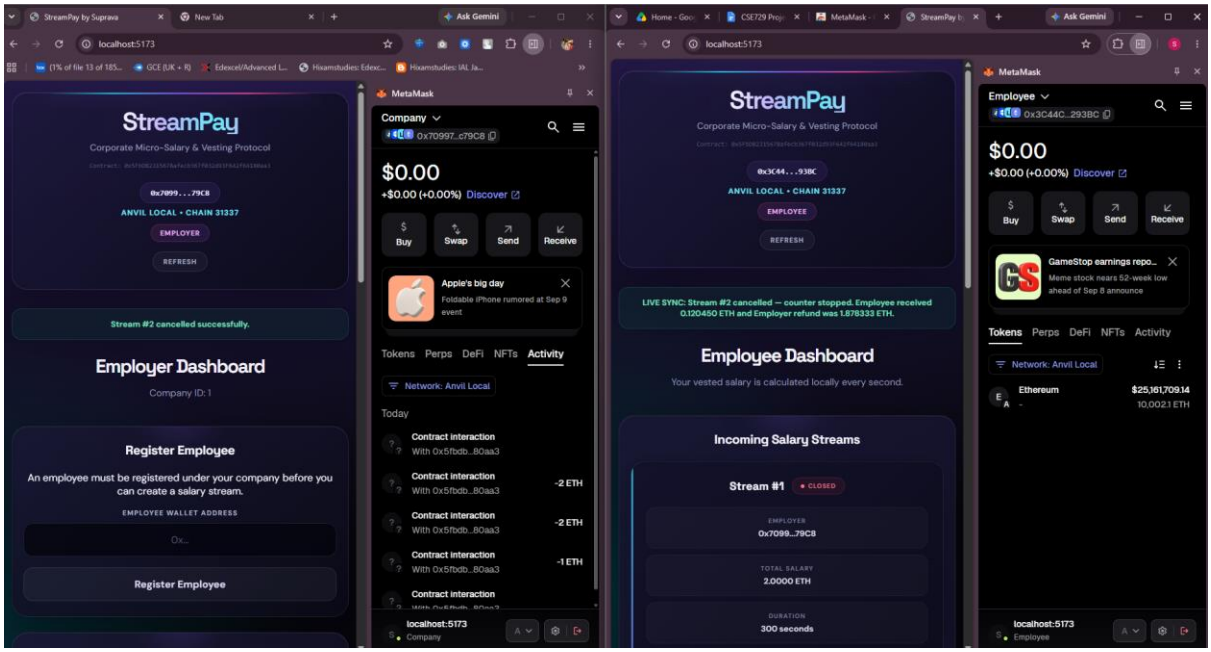

Figure 33 Live Stream Cancellation Notification in both the Employer and the Employee

In the following Figure 34, the Company dashboard displays the amount of unvested ETH returned to the Employer after the stream was cancelled.

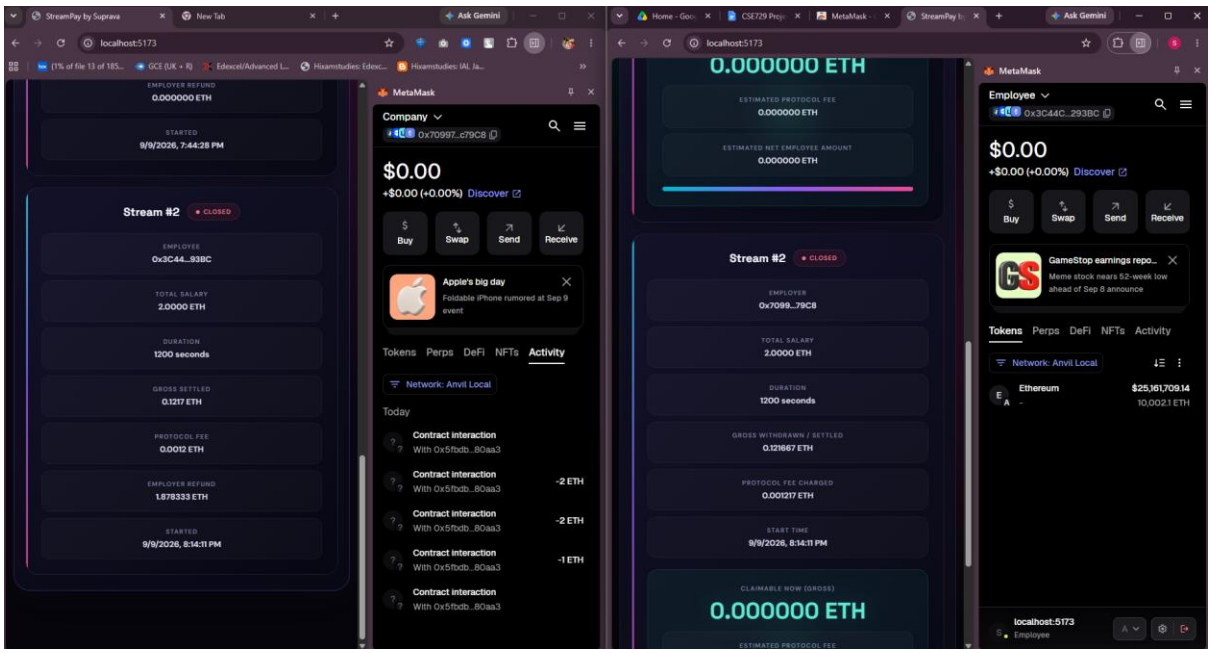

Figure 34 Employer Refund Amount Shown in the Company Dashboard

19. Testing and Validation

The project was validated using Foundry automated testing.

Final Result:

13 Tests Passed

0 Failed

0 Skipped

Testing covered:

- Admin deployment.
- Employee registration.
- Stream creation.
- Vesting calculation.
- Withdrawal.
- Protocol fees.
- Multiple withdrawals.
- Cancellation.
- Refund handling.
- Unauthorized operations.

The StreamPay smart contract was validated using Foundry's automated testing framework. The final test execution completed successfully with **13 tests passed, 0 failed, and 0 skipped**.

The tests covered the major functional and security requirements of the protocol, including administrator initialization, employee registration, stream creation, time-based vesting calculation, employee withdrawal, protocol fee accounting, multiple withdrawals, stream cancellation, employer refund handling, and prevention of unauthorized operations.

The successful test results provide additional verification that the core smart contract functions operate according to the specified rules and that previously withdrawn funds cannot be claimed again.

20. Security Considerations

Security features include:

- Wallet-based authentication.
- Role validation.
- Employee registration checks.
- Withdrawal accounting.
- Unauthorized cancellation prevention.
- Transparent blockchain records.

21. Challenges and Solutions

Challenge 1: Synchronizing frontend and blockchain state

One challenge was keeping the frontend interface consistent with the latest smart contract state after transactions such as stream creation, withdrawal, and cancellation. A transaction can remain pending before the blockchain confirms it.

Solution: The application uses transaction confirmation through tx.wait() before updating the relevant interface state. This ensures that the frontend reflects the confirmed blockchain state rather than assuming that a submitted transaction has already succeeded.

Challenge 2: Real-time vesting display

The Employee dashboard needed to show the increasing vested salary every second. Querying the blockchain through RPC every second would create unnecessary network requests and introduce latency.

Solution: The required stream state is fetched from the smart contract once, after which a client-side JavaScript interval calculates the unlocked amount locally every second using the same integer-based vesting formula as the smart contract.

Challenge 3: Fair stream cancellation

Cancellation required correctly separating the employee's earned amount from the employer's remaining locked balance. An incorrect calculation could result in either overpayment to the employee or an incorrect employer refund.

Solution: The cancellation logic calculates the vested amount first, settles the employee's earned balance after the protocol fee, and returns the remaining unvested balance to the employer. The stream is then marked as closed to prevent further withdrawals or cancellation.

22. Future Improvements

Future improvements may include:
• Mainnet deployment.
• ERC-20 token salary support.
• Stablecoin payroll.
• Mobile application.
• Multi-company payroll management.
• Advanced analytics dashboard.

23. Conclusion

StreamPay successfully demonstrates a decentralized payroll streaming system using blockchain technology. The project achieves automated salary vesting, transparent payment processing, protocol fee management, and fair cancellation settlement.

The combination of smart contract logic, wallet integration, frontend dashboards, and automated testing demonstrates a complete blockchain application workflow.